MUST SMART CAMPUS GUIDE SYSTEM

(MSCGS)

Software Project Documentation

A digital campus navigation and information system for
Mbeya University of Science and Technology (MUST)

Prepared for: Portfolio / Academic Software Engineering Project
Institution: Mbeya University of Science and Technology (MUST)
Document Version: 1.0
Date: July 2026

Table of Contents

1. Executive Summary

In one sentence: MSCGS is a "Google Maps for MUST campus" — a simple app where anyone can type a place name (like "Library" or "Computer Laboratory") and instantly see what it is, where it is, and how to walk there.

The problem it solves is small but universal: every semester, hundreds of new students and visitors lose time wandering around campus because there is no single place to look up a location. MSCGS fixes this with three simple building blocks — a **map**, a **search box**, and a **directions feature** — backed by a clean, well-structured database and API.

This project is intentionally scoped so it can be built in manageable phases: start with a working backend that stores and serves location data, then add the map and search, then add navigation and a mobile app. Each phase produces something demonstrable — which makes it ideal both as a learning project and as a portfolio piece.

2. Project Overview

The MUST Smart Campus Guide System (MSCGS) is a digital platform designed to help students, staff, and visitors of Mbeya University of Science and Technology (MUST) easily locate places within the campus. The system provides an interactive campus map, a location search engine, detailed information about buildings and offices, and step-by-step directions to help users navigate the university grounds with confidence.

The system is intended to serve as a central, reliable reference point for anyone moving around campus for the first time — reducing confusion, saving time, and improving the overall experience of new students, staff, and guests.

In simple terms: instead of asking around or getting lost, a user opens the app, types where they want to go, and the system shows them exactly where it is and how to get there.

3. Problem Statement

Life on campus presents recurring navigation challenges for people who are unfamiliar with the layout of MUST:

- New students often struggle to identify the location of key facilities on campus.
- Students waste valuable time searching for lecture rooms, laboratories, and offices.
- Visitors are frequently unaware of where to go to access specific services.
- There is no single, easy-to-use system that guides users to the correct location.

Example scenario: A first-year student is looking for the "Computer Engineering Laboratory" but has no way of knowing where it is located on campus.

Why this matters: these small delays add up — missed classes, late arrivals to exams, and a stressful first impression of the university for new students and guests. Solving this with software is low-cost compared to physical signage, and it can be updated instantly whenever a building or office changes.

4. Main Objective

To design and develop a system that enables students and visitors to locate various places within MUST easily, accurately, and quickly.

5. Specific Objectives

The system shall be able to:

- Display an interactive map of the campus.
- Allow users to search for a specific location.
- Provide detailed information about each building or office.
- Provide directions from one point to another within campus.
- Help new students familiarize themselves with the campus environment.

Each objective above maps directly to a feature described later in Section 7 — this keeps the project focused: nothing is built that doesn't serve one of these five goals.

6. System Users & Roles

The system is built around three types of people. Separating them clearly from the start makes the permissions system (who can edit vs. who can only view) simple to design and secure.

Role	Capabilities
Student	Search for locations, view the campus map, get directions, view office/facility information.
Administrator	Add new locations, update existing information, delete locations, manage categories and feedback.
Visitor	View key campus locations and public information without needing to register an account.

7. Core Features (Version 1)

"Version 1" means the minimum set of features needed for the system to be genuinely useful on day one. Extra features (like crowd-sourced reviews or indoor navigation) can be added later without changing this foundation.

A. Campus Map

The map is the visual entry point of the app — most users will open the app and simply look at the map before they even search for anything. It shows building markers, including:

- Main Gate
- Library
- ICT Building
- Engineering Block
- Administration Block

B. Location Search

Search is the fastest path to an answer — a user who already knows the name of a place doesn't need to browse the map at all; they just type it. Example search interaction:

Field	Value
Input	"Library"
Name	University Library
Category	Academic Facility
Location	Engineering Campus
Description	Place for books and research materials

C. Location Details

Once a user finds a place, they need enough detail to actually use it — not just a pin on a map. Each location record shall include:

- **Name** — what the place is called.
- **Category** — what kind of place it is (office, lab, hall, accommodation, etc).
- **Description** — a short, plain explanation of what happens there.
- **Photo** — a picture so the user recognizes the building on sight.
- **Coordinates (latitude, longitude)** — the exact GPS position, used to place it on the map and calculate directions.
- **Working hours** — so users know when the place is actually open.

D. Navigation

Navigation turns "where is it" into "how do I get there" — this is the feature that saves the most time for someone completely unfamiliar with campus. Example navigation output:

Field	Value
From	Main Gate
To	Computer Laboratory
Direction	Step-by-step walking route
Distance	Calculated in meters/kilometers
Estimated walking time	Calculated in minutes

8. Technology Stack

Layer	Technology
Backend	FastAPI, PostgreSQL, SQLAlchemy, JWT Authentication
Frontend (Option 1)	React (web application)
Frontend (Option 2)	Flutter (mobile application)
Map	OpenStreetMap data with Leaflet.js for rendering

9. Why This Stack? (Explained Simply)

Choice	Plain-language reason
FastAPI	A modern Python web framework that is fast to write in, fast to run, and automatically generates interactive API documentation (Swagger UI) for testing every endpoint.
PostgreSQL	A reliable, production-grade relational database — well suited to structured data like locations, categories, and users, and capable of storing geographic coordinates.
SQLAlchemy	An ORM (Object-Relational Mapper) that lets the database tables be written and used as plain Python classes, instead of writing raw SQL everywhere.
JWT Authentication	A secure, industry-standard way to confirm who is logged in (e.g. an Administrator) without storing session state on the server.
React / Flutter	Two options for the interface — React for a website, Flutter for a mobile app that works on both Android and iOS from one codebase.
OpenStreetMap + Leaflet.js	A free, open map data source and a lightweight library to display it — no paid map API keys required, which keeps the project free to run and easy to deploy.

10. Database Design (Initial)

The database has four tables. Each one stores one clear "thing" — this separation keeps the system easy to maintain and extend later (for example, adding a new category doesn't require touching the locations table at all).

Table: users

Field	Type / Notes
id	Primary key, integer
name	String — full name of user
email	String, unique — login identifier
password	String — hashed password
role	Enum — student / administrator / visitor

Table: locations

Field	Type / Notes
id	Primary key, integer
name	String — name of the location
category	Foreign key → categories.id
description	Text — details about the location
latitude	Float — GPS coordinate
longitude	Float — GPS coordinate
image	String — image path or URL

Table: categories

Field	Type / Notes
id	Primary key, integer
name	String — e.g. Office, Laboratory, Lecture Hall, Accommodation

Table: feedback

Field	Type / Notes
id	Primary key, integer
user_id	Foreign key → users.id
location_id	Foreign key → locations.id

message	Text — feedback comment
rating	Integer — numeric rating score

11. Entity Relationship Overview

The core entities and their relationships can be summarized as follows:

- **users** (1) —— (many) **feedback**: a user can submit multiple feedback entries.
- **locations** (1) —— (many) **feedback**: a location can receive multiple feedback entries.
- **categories** (1) —— (many) **locations**: a category can classify multiple locations.

This structure keeps the schema normalized: categories are managed independently from locations, and feedback links both a user and a location, enabling ratings and comments to be traced back to their source. A full ERD diagram (with cardinality notation) is a recommended next artifact once the schema is finalized in code.

Simple text diagram

```
categories (1) <---- (many) locations (1) <---- (many) feedback (many) ----> (1) users
```

Read left to right: one category groups many locations; one location can receive many feedback entries; and each feedback entry belongs to exactly one user.

12. Non-Functional Requirements

Beyond "what the system does" (functional requirements, covered above), a professional specification also states "how well" it should do it:

- **Performance:** a location search should return results in under 1 second on a normal campus Wi-Fi or mobile data connection.
- **Usability:** a first-time user (e.g. a new student) should be able to find a location without any training or instructions.
- **Availability:** the system should be accessible during all university operating hours, with minimal downtime for maintenance.
- **Security:** only authenticated Administrators can create, edit, or delete location records; passwords are never stored in plain text.
- **Scalability:** the database design should comfortably support future growth — more buildings, more categories, and more users — without redesign.
- **Maintainability:** code is organized into clear layers (models, schemas, services, routers) so new features can be added without breaking existing ones.

13. Example Use Case Walkthrough

To make the system concrete, here is a full walkthrough of the exact scenario introduced in the Problem Statement:

1. A first-year student opens the MSCGS app for the first time.
2. They see the campus map with markers for the Main Gate, Library, ICT Building, Engineering Block, and Administration Block.
3. They type "Computer Engineering Laboratory" into the search box.
4. The system returns the location's name, category, description, and coordinates.
5. The student taps "Get Directions" and sets their current position as "Main Gate".
6. The system calculates and displays the walking route, distance, and estimated time.
7. The student follows the route and arrives at the correct building without asking anyone for help.

This single walkthrough touches every core feature in Section 7 — the map, search, location details, and navigation — which is a useful way to demonstrate the finished product later.

14. Development Plan / Roadmap

Phase	Deliverables
Phase 1	Database design, FastAPI project setup, Location CRUD API
Phase 2	Map integration, search system
Phase 3	Navigation feature, mobile application

15. Glossary of Terms

Plain-language definitions for the technical terms used throughout this document:

Term	Meaning
API	Application Programming Interface — the set of rules that lets the frontend (app/website) talk to the backend (server).
CRUD	Create, Read, Update, Delete — the four basic operations performed on data.
ORM	Object-Relational Mapper — lets developers work with database tables as ordinary code objects.
JWT	JSON Web Token — a small, secure piece of data used to confirm a user's identity after login.
ERD	Entity Relationship Diagram — a visual map of database tables and how they connect.
Endpoint	A specific URL the app can call to perform an action, e.g. "search for a location".
Coordinates	A pair of numbers (latitude, longitude) that pinpoint an exact spot on Earth.
Schema	The structure/shape of the data — which fields exist and what type each one is.

16. Next Steps

- Draw the full ERD (entity relationship diagram) with cardinality notation.
- Set up the FastAPI project structure (routers, models, schemas, services, core config).
- Implement the SQLAlchemy database models for locations, categories, users, and feedback.

This document combines Backend development, Database design, Map integration, and real-world problem solving — making it a strong, well-rounded portfolio project.